CS305 – Programming Languages

Fall 2025-2026

HOMEWORK 5

Implementing a Scheme interpreter

Due date: May 22nd, 2026 @ 23:55

NOTE

Only SUCourse submission is allowed. No submission by e-mail. Please see the note at the end of this document for late submission policy.

1 Introduction

In this homework you will implement a Scheme interpreter, similar to the ones we saw in the lectures. However, the subset of Scheme that is handled by the interpreter will be different.

2 The Scheme subset s7

The syntax of the Scheme subset that will be covered by the interpreter that you will implement for this homework is given in this section.

Grammar for the subset **s7**

```
<s7>                    -> <define>
                        |  <expr>

<define>                -> ( define IDENT <expr> )

<expr>                  -> NUMBER
                        |  IDENT
                        |  <let>
                        |  <letstar>
                        |  <cond>
                        |  <if>
                        |  ( <operator> <operand_list> )

<operator>              -> + | * | - | /

<let>                   -> ( let ( <var_binding_list> ) <expr> )

<letstar>               -> ( let* ( <var_binding_list> ) <expr> )

<cond>                  -> ( cond ( <expression_pair_list> ) )

<if>                    -> ( if <expr> <expr> <expr> )

<expression_pair_list> -> ( <expr>  <expr> ) <expression_pair_list>
                        |  ( else <expr> )
                        |

<operand_list>          -> <expr> <operand_list>
                        |

<var_binding_list>      -> ( IDENT <expr> ) <var_binding_list>
                        |
```

The s7 language has been extended to support a dynamic conditional cond construct, an if construct, and local variable bindings.

The cond expression takes the form (cond (clauses)). In this implementation, each clause within the cond contains an evaluatable test expression and an associated result expression. Execution proceeds by iterating sequentially through the clauses. For each clause (test-expr result-expr), the test-expr is evaluated in the current environment. A clause is considered to match if its test-expr evaluates to a non-zero value (any number other than 0 is treated as true; 0 is treated as false). Upon finding the first matching clause, the corresponding result-expr is evaluated and returned, and no further clauses are examined. If an else clause is present, its expression is evaluated if no previous matches are found. If no clause matches and there is no else clause, the interpreter should produce an error.

The if expression takes the form (if test-expr then-expr else-expr). The test-expr is evaluated first in the current environment. If it evaluates to a non-zero value (treated as true), the then-expr is evaluated and its value is returned; the else-expr is not evaluated in this case. Otherwise (i.e., if the test evaluates to 0), the else-expr is evaluated and its value is returned, while the then-expr is not evaluated. Both branches must be present; an if expression with a missing branch is a syntax error.

Note that anything accepted as a number by the "number?" predicate is a number for our interpreter. Variables can be globally defined using the define construct. If an identifier is used that has not been defined in the global or local environment, an error should be displayed.

Variable scoping is handled through let and let* expressions:

- **let**: Used for parallel binding. All expressions in the <var_binding_list> are evaluated in the environment outside the let before any variables are bound. A let expression cannot have multiple bindings for the same variable; if it does, an error must be produced.
- **let***: Used for sequential binding. Each expression in the <var_binding_list> is evaluated and immediately bound to its identifier. This allows subsequent bindings in the same list to reference variables defined earlier in that same list.

Regarding the arithmetic operators (+, *, -, /):

- You can assume that the <operand_list> will always contain two or more items.
- The division and subtraction operators are left-associative.
- You do not need to check for division by zero.

Your interpreter should check the syntax of the expression. If the syntax is incorrect or a variable is undefined, an error message should be displayed, and the interpreter

should return to the prompt.

Below, we provide some sample interactions in "Scheme Interaction" part, which we hope would explain the semantics of the constructs a little bit more. You can also see the format of the error and value messages to be displayed in this part.

3 The procedure `cs305`

You should declare a procedure named `cs305` which will start the interpretation when called. It should not take any arguments.

In every iteration of your REPL, you should print out the prompt given below in "Scheme Interaction" sample, then accept an input from the user, then evaluate the value of the input expression, and finally print the value evaluated by using a value prompt. The following is a sample on how the interaction with your interpreter must look like.

Scheme Interaction

```
1 ]=> (cs305)
cs305> 3
cs305: 3

cs305> (define x 5)
cs305: x

cs305> x
cs305: 5

cs305> (define y 10)
cs305: y

cs305> (+ x y 2)
cs305: 17

cs305> (let ( (x 2) (y x) ) (+ x y) )
cs305: 7

cs305> (let* ( (x 2) (y x) ) (+ x y) )
cs305: 4

cs305> (define z (let ( (a 10) (b 20) ) (* a b) ) )
cs305: z

cs305> z
cs305: 200

cs305> (let ( (x 1) (x 2) ) (+ x 1) )
cs305: ERROR

cs305> (let ((x)) (+ x y))

cs305: ERROR

cs305>  (let (()) (+ x y))
cs305: ERROR

cs305> (cond ( (0 100) (5 200) ) )
cs305: 200

cs305> (cond ( ((- x 5) 10) (1 50) ) )
cs305: 50

cs305> (cond ( (0 1) (0 2) (else (+ 5 5)) ) )
cs305: 10
```

Scheme Interaction cont'd

```
cs305> (define target 7)
cs305: target

cs305> (cond ( ( (- target 7) 1) (else 0) ) )
cs305: 0

cs305> (cond ( (0 1) (0 2) ) )
cs305: ERROR

cs305> (if 1 100 200)
cs305: 100

cs305> (if 0 100 200)
cs305: 200

cs305> (if (- x 5) 100 200)
cs305: 200

cs305> (if (+ 1 1) (let ( (a 3) (b 4) ) (* a b) ) 999)
cs305: 12

cs305> (if 1 5)
cs305: ERROR

cs305> (let* ( (x 1) (y (+ x 1) ) (z (* y 2) ) ) (+ x y z) )
cs305: 7

cs305> (define 1 y)
cs305: ERROR

cs305> (def x 1)
cs305: ERROR

cs305> undefined_var
cs305: ERROR
```

As you may recall, when an error is detected, the MIT Scheme Interpreter and the interpreters we developed in the class, produce an error message and go into an error prompt.

However, in this homework, as you may have noticed in the interaction given above, when there is an error, you interpreter should display an error message and **it should go back to "the read prompt" again**, not into another error prompt. Hence, we will be able to interact with the interpreter even after there is an error in the expression.

In order to do this, you can simply use the built-in "display" procedure to display error messages (not the "error" procedure we used in the interpreters we developed in the class).

4 How to Submit

Submit your Scheme file named as `username-hw5.scm` where `username` is your SUCourse username. In this file you must have the following Scheme procedure defined:

```
(define cs305 (lambda () ...  ))
```

where the part shown as `...` represents your code that you will have there.

Of course, you will have many other procedures in your Scheme file, but `cs305` is the procedure that we will use.

5 Notes

- **Important**: Name your files as you are told and **don't zip them.** [-10 points otherwise]
- **Important: Make sure your procedure name is exactly cs305**
- **Important: Since this homework is evaluated automatically make sure your output is exactly as it is supposed to be.**
- You may get help from our TA or from your friends. However, **you must implement the homework by yourself**.
- Start working on the homework immediately.
- If you develop your code or create your test files on your own computer (not on cs305.sabanciuniv.edu), there can be incompatibilities once you transfer them to the cs305 machine. Since the grading will be done automatically on the cs305 machine, we strongly encourage you to do your development on the cs305 machine, or at least test your code on the cs305 machine before submitting it. If you prefer not to test your implementation on the cs305 machine, this means you accept to take the risks of incompatibilities. Even if you may have spent hours on the homework, you can easily get 0 due to such incompatibilities.

LATE SUBMISSION POLICY

Late submission is allowed subject to the following conditions:

- Your homework grade will be decided by multiplying what you get from the test cases by a "submission time factor (STF)".
- If you submit on time (i.e. before the deadline), your STF is 1. So, you don't lose anything.
- If you submit late, you will lose 0.01 of your STF for every 5 mins of delay.
- We will not accept any homework later than 500 mins after the deadline.
- SUCourse's timestamp will be used for STF computation.
- If you submit multiple times, the last submission time will be used.